Procesamiento Masivo de Datos

Repaso de Pthreads, OpenMP, MPI y modelos híbridos
Camino más corto por fuerza bruta

Alex Di Genova

Universidad de O'Higgins

Objetivos

- Resolver el mismo problema con seis implementaciones en C.

Objetivos

- Resolver el mismo problema con seis implementaciones en C.
- Repasar creación de hilos, regiones paralelas y comunicación colectiva.

Objetivos

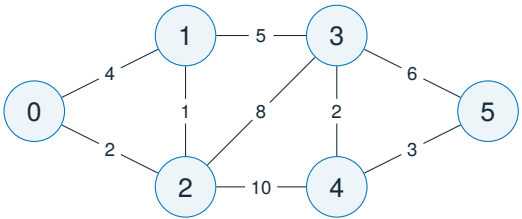
- Resolver el mismo problema con seis implementaciones en C.
- Repasar creación de hilos, regiones paralelas y comunicación colectiva.
- Comparar reparto, memoria y sincronización.

Objetivos

- Resolver el mismo problema con seis implementaciones en C.
- Repasar creación de hilos, regiones paralelas y comunicación colectiva.
- Comparar reparto, memoria y sincronización.
- Diseñar experimentos y completar ejercicios.

El problema

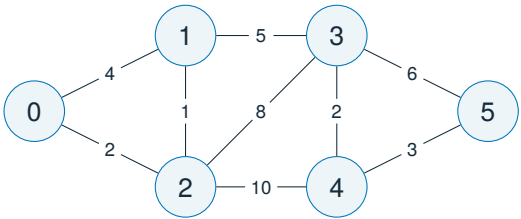
Camino de menor peso entre 0 y 5



- Grafo no dirigido, pesos positivos.

El problema

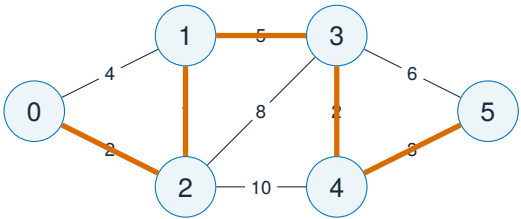
Camino de menor peso entre 0 y 5



- Grafo no dirigido, pesos positivos.
- Enumerar caminos simples: ningún vértice se repite.

El problema

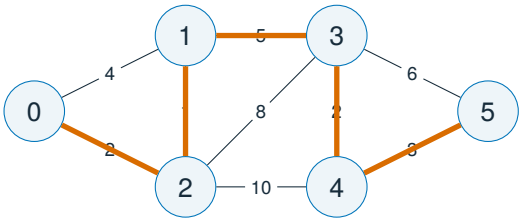
Camino de menor peso entre 0 y 5



- Grafo no dirigido, pesos positivos.
- Enumerar caminos simples: ningún vértice se repite.
- Óptimo: $0 \to 2 \to 1 \to 3 \to 4 \to 5$.

El problema

Camino de menor peso entre 0 y 5



- Grafo no dirigido, pesos positivos.
- Enumerar caminos simples: ningún vértice se repite.
- Óptimo: $0 \to 2 \to 1 \to 3 \to 4 \to 5$.
- Distancia: 2 + 1 + 5 + 2 + 3 = 13.

Fuerza bruta y descomposición

- DFS explora recursivamente vecinos no visitados; al llegar al destino compara el costo.

Costo y límites

El número de caminos simples puede crecer factorialmente. Para pesos no negativos, Dijkstra es una alternativa mucho más eficiente.

Fuerza bruta y descomposición

- DFS explora recursivamente vecinos no visitados; al llegar al destino compara el costo.
- Una tarea por posible primer vecino de 0; si no hay arista produce infinito.

Costo y límites

El número de caminos simples puede crecer factorialmente. Para pesos no negativos, Dijkstra es una alternativa mucho más eficiente.

Fuerza bruta y descomposición

- DFS explora recursivamente vecinos no visitados; al llegar al destino compara el costo.
- Una tarea por posible primer vecino de 0; si no hay arista produce infinito.
- Cada tarea mantiene su camino, visitados y mejor resultado.

Costo y límites

El número de caminos simples puede crecer factorialmente. Para pesos no negativos, Dijkstra es una alternativa mucho más eficiente.

Fuerza bruta y descomposición

- DFS explora recursivamente vecinos no visitados; al llegar al destino compara el costo.
- Una tarea por posible primer vecino de 0; si no hay arista produce infinito.
- Cada tarea mantiene su camino, visitados y mejor resultado.
- Se combinan resultados por mínimo; en caso de empate puede devolverse cualquier óptimo.

Costo y límites

El número de caminos simples puede crecer factorialmente. Para pesos no negativos, Dijkstra es una alternativa mucho más eficiente.

Núcleo común: búsqueda DFS

```
static void dfs(int u, int target, unsigned visited, int cost,
                int path[N], int length, Result *best) {
    if (u == target) {
        if (cost < best->distance) {
            best->distance = cost; best->length = length;
            for (int i = 0; i < length; ++i) best->path[i] = path[i];
        }
        return;
    }
    for (int v = 0; v < N; ++v)
        if (graph[u][v] && !(visited & (1u << v))) {
            path[length] = v;
            dfs(v, target, visited | (1u << v), cost + graph[u][v],
                path, length + 1, best);
        }
}
```

Versión serial

Una tarea a la vez

```
#include "common.h"
int main(void) {
    Result best = empty_result();
    for (int i = 1; i < N; ++i)
        combine(&best, search_task(i));
    print_result(best);
    return 0;
}
```

- `search_task(i)` explora los caminos que comienzan por $0 \to i$.

Versión serial

Una tarea a la vez

```
#include "common.h"
int main(void) {
    Result best = empty_result();
    for (int i = 1; i < N; ++i)
        combine(&best, search_task(i));
    print_result(best);
    return 0;
}
```

- `search_task(i)` explora los caminos que comienzan por $0 \to i$.
- `combine` conserva el resultado con menor distancia.

Repaso: Pthreads

- Los hilos comparten memoria; cada uno tiene pila propia.

Repaso: Pthreads

- Los hilos comparten memoria; cada uno tiene pila propia.
- `pthread_create` inicia una función con un argumento.

Repaso: Pthreads

- Los hilos comparten memoria; cada uno tiene pila propia.
- `pthread_create` inicia una función con un argumento.
- `pthread_join` espera el término y recupera su resultado.

Repaso: Pthreads

- Los hilos comparten memoria; cada uno tiene pila propia.
- `pthread_create` inicia una función con un argumento.
- `pthread_join` espera el término y recupera su resultado.
- Evitamos carreras con resultados privados, combinados después de `join`.

Repaso: Pthreads

- Los hilos comparten memoria; cada uno tiene pila propia.
- `pthread_create` inicia una función con un argumento.
- `pthread_join` espera el término y recupera su resultado.
- Evitamos carreras con resultados privados, combinados después de `join`.
- Crear hilos tiene costo; un conjunto persistente de trabajadores puede reducirlo.

Pthreads: función trabajadora

```
static void *worker(void *arg) {
    int i = *(int *)arg;
    Result *r = malloc(sizeof *r);
    if (r) *r = search_task(i);
    return r;
}
```

- El argumento apunta a un identificador estable.

Pthreads: función trabajadora

```
static void *worker(void *arg) {
    int i = *(int *)arg;
    Result *r = malloc(sizeof *r);
    if (r) *r = search_task(i);
    return r;
}
```

- El argumento apunta a un identificador estable.
- El resultado reservado se recupera tras `join` y luego se libera.

Pthreads: crear y esperar

```
    for (int i = base; i < N && count < THREADS; ++i) {
        ids[count] = i;
        if (pthread_create(&threads[count], NULL, worker, &ids[count])) {
            fputs("Error al crear hilo\n", stderr); exit(EXIT_FAILURE);
        }
        ++count;
    }
```

```
    for (int j = 0; j < count; ++j) {
        void *ptr;
        if (pthread_join(threads[j], &ptr) || !ptr) {
            fputs("Error en hilo\n", stderr); exit(EXIT_FAILURE);
        }
        Result *r = ptr;
        combine(&best, *r);
        free(r);
    }
```

Se procesan tareas en lotes de hasta tres hilos y se verifican las llamadas.

Repaso: OpenMP

- Directivas de compilador para paralelismo de memoria compartida.

Repaso: OpenMP

- Directivas de compilador para paralelismo de memoria compartida.
- `parallel for` distribuye iteraciones entre hilos.

Repaso: OpenMP

- Directivas de compilador para paralelismo de memoria compartida.
- `parallel for` distribuye iteraciones entre hilos.
- `schedule(dynamic)` entrega iteraciones a medida que terminan: útil con ramas de tamaño variable.

Repaso: OpenMP

- Directivas de compilador para paralelismo de memoria compartida.
- `parallel for` distribuye iteraciones entre hilos.
- `schedule(dynamic)` entrega iteraciones a medida que terminan: útil con ramas de tamaño variable.
- Cada iteración escribe una posición distinta; una barrera implícita precede a la combinación serial.

OpenMP: distribuir las tareas

```
#pragma omp parallel for schedule(dynamic)
for (int i = 1; i < N; ++i)
    results[i] = search_task(i);
/* Barrera implicita al salir de parallel for. */
Result best = empty_result();
for (int i = 1; i < N; ++i)
    combine(&best, results[i]);
```

- `results[i]` es independiente para cada iteración.

OpenMP: distribuir las tareas

```
#pragma omp parallel for schedule(dynamic)
for (int i = 1; i < N; ++i)
    results[i] = search_task(i);
/* Barrera implicita al salir de parallel for. */
Result best = empty_result();
for (int i = 1; i < N; ++i)
    combine(&best, results[i]);
```

- `results[i]` es independiente para cada iteración.
- El número de hilos se configura con `OMP_NUM_THREADS`.

Repaso: MPI

- Cada proceso tiene memoria propia y un rango en un comunicador.

Repaso: MPI

- Cada proceso tiene memoria propia y un rango en un comunicador.
- `MPI_Init`, `MPI_Comm_rank`, `MPI_Comm_size` y `MPI_Finalize` gestionan la ejecución.

Repaso: MPI

- Cada proceso tiene memoria propia y un rango en un comunicador.
- `MPI_Init`, `MPI_Comm_rank`, `MPI_Comm_size` y `MPI_Finalize` gestionan la ejecución.
- Reparto cíclico: rango r procesa $i = 1 + r, 1 + r + P, \ldots$

Repaso: MPI

- Cada proceso tiene memoria propia y un rango en un comunicador.
- `MPI_Init`, `MPI_Comm_rank`, `MPI_Comm_size` y `MPI_Finalize` gestionan la ejecución.
- Reparto cíclico: rango r procesa $i = 1 + r, 1 + r + P, \ldots$
- Colectivas: todos los procesos del comunicador deben llamarlas en el mismo orden.

MPI: reparto entre procesos

```
MPI_Init(&argc, &argv);
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
Result local = empty_result();
for (int i = 1 + rank; i < N; i += size)
    combine(&local, search_task(i));
collect(local, rank);
MPI_Finalize();
```

MPI: reparto entre procesos

```
MPI_Init(&argc, &argv);
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
Result local = empty_result();
for (int i = 1 + rank; i < N; i += size)
    combine(&local, search_task(i));
collect(local, rank);
MPI_Finalize();
```

Cada proceso calcula un mínimo local; el grafo está definido idénticamente en cada proceso.

MPI: elegir el camino ganador

```
MPI_Allreduce(&in, &out, 1, MPI_2INT, MPI_MINLOC, MPI_COMM_WORLD);
MPI_Bcast(&local.length, 1, MPI_INT, out.rank, MPI_COMM_WORLD);
MPI_Bcast(local.path, local.length, MPI_INT, out.rank, MPI_COMM_WORLD);
```

- `MPI_MINLOC` obtiene el mínimo y rango ganador; desempata por menor rango.

MPI: elegir el camino ganador

```
MPI_Allreduce(&in, &out, 1, MPI_2INT, MPI_MINLOC, MPI_COMM_WORLD);
MPI_Bcast(&local.length, 1, MPI_INT, out.rank, MPI_COMM_WORLD);
MPI_Bcast(local.path, local.length, MPI_INT, out.rank, MPI_COMM_WORLD);
```

- `MPI_MINLOC` obtiene el mínimo y rango ganador; desempata por menor rango.
- El ganador difunde longitud y camino a todos; el rango 0 imprime.

MPI: elegir el camino ganador

```
MPI_Allreduce(&in, &out, 1, MPI_2INT, MPI_MINLOC, MPI_COMM_WORLD);
MPI_Bcast(&local.length, 1, MPI_INT, out.rank, MPI_COMM_WORLD);
MPI_Bcast(local.path, local.length, MPI_INT, out.rank, MPI_COMM_WORLD);
```

- `MPI_MINLOC` obtiene el mínimo y rango ganador; desempata por menor rango.
- El ganador difunde longitud y camino a todos; el rango 0 imprime.
- Sin camino, la distancia es infinito y la longitud es cero.

Modelos híbridos

Procesos MPI $\times$ hilos por proceso

- MPI distribuye tareas entre procesos; cada proceso utiliza hilos localmente.

Modelos híbridos

Procesos MPI $\times$ hilos por proceso

- MPI distribuye tareas entre procesos; cada proceso utiliza hilos localmente.
- `MPI_Init_thread` solicita `MPI_THREAD_FUNNELED`: solo el hilo que inicializó MPI puede llamar a MPI.

Modelos híbridos

Procesos MPI $\times$ hilos por proceso

- MPI distribuye tareas entre procesos; cada proceso utiliza hilos localmente.
- `MPI_Init_thread` solicita `MPI_THREAD_FUNNELED`: solo el hilo que inicializó MPI puede llamar a MPI.
- Se verifica el nivel proporcionado; las colectivas ocurren después de esperar a los hilos.

Modelos híbridos

Procesos MPI $\times$ hilos por proceso

- MPI distribuye tareas entre procesos; cada proceso utiliza hilos localmente.
- `MPI_Init_thread` solicita `MPI_THREAD_FUNNELED`: solo el hilo que inicializó MPI puede llamar a MPI.
- Se verifica el nivel proporcionado; las colectivas ocurren después de esperar a los hilos.
- Procesos $\times$ hilos debe respetar los recursos asignados.

MPI + OpenMP: inicialización

```
int provided;
MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
if (provided < MPI_THREAD_FUNNELED) MPI_Abort(MPI_COMM_WORLD, 1);
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

MPI + OpenMP: inicialización

```
int provided;
MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
if (provided < MPI_THREAD_FUNNELED) MPI_Abort(MPI_COMM_WORLD, 1);
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

Todos los procesos solicitan y verifican el soporte antes de hacer trabajo paralelo.

MPI + OpenMP: dos niveles de reparto

```
#pragma omp parallel for schedule(dynamic)
for (int i = 1; i < N; ++i)
    if ((i - 1) % size == rank) results[i] = search_task(i);
Result local = empty_result();
for (int i = 1 + rank; i < N; i += size)
    combine(&local, results[i]);
/* Solo el hilo principal llama a MPI, fuera de la region paralela. */
collect(local, rank);
MPI_Finalize();
```

MPI + OpenMP: dos niveles de reparto

```
#pragma omp parallel for schedule(dynamic)
for (int i = 1; i < N; ++i)
    if ((i - 1) % size == rank) results[i] = search_task(i);
Result local = empty_result();
for (int i = 1 + rank; i < N; i += size)
    combine(&local, results[i]);
/* Solo el hilo principal llama a MPI, fuera de la region paralela. */
collect(local, rank);
MPI_Finalize();
```

El rango selecciona tareas; OpenMP distribuye iteraciones. Después de la barrera, el hilo inicial llama a MPI.

MPI + Pthreads: trabajo local

```
static void *worker(void *arg) {
    Work *w = arg;
    w->best = empty_result();
    for (int i = 1 + w->rank; i < N; i += w->size)
        if (((i - 1 - w->rank) / w->size) % THREADS == w->id)
            combine(&w->best, search_task(i));
    return NULL;
}
```

MPI + Pthreads: trabajo local

```
static void *worker(void *arg) {
    Work *w = arg;
    w->best = empty_result();
    for (int i = 1 + w->rank; i < N; i += w->size)
        if (((i - 1 - w->rank) / w->size) % THREADS == w->id)
            combine(&w->best, search_task(i));
    return NULL;
}
```

Cada hilo recorre una parte de las tareas del proceso y mantiene su propio mínimo.

MPI + Pthreads: coordinación

```
pthread_t threads[THREADS]; Work work[THREADS];
for (int t = 0; t < THREADS; ++t) {
    work[t] = (Work){.id=t, .rank=rank, .size=size};
    if (pthread_create(&threads[t], NULL, worker, &work[t]))
        MPI_Abort(MPI_COMM_WORLD, 2);
}
Result local = empty_result();
for (int t = 0; t < THREADS; ++t) {
    if (pthread_join(threads[t], NULL)) MPI_Abort(MPI_COMM_WORLD, 3);
    combine(&local, work[t].best);
}
collect(local, rank); /* Solo el hilo inicial llama a MPI. */
MPI_Finalize();
```

MPI + Pthreads: coordinación

```
pthread_t threads[THREADS]; Work work[THREADS];
for (int t = 0; t < THREADS; ++t) {
    work[t] = (Work){.id=t, .rank=rank, .size=size};
    if (pthread_create(&threads[t], NULL, worker, &work[t]))
        MPI_Abort(MPI_COMM_WORLD, 2);
}
Result local = empty_result();
for (int t = 0; t < THREADS; ++t) {
    if (pthread_join(threads[t], NULL)) MPI_Abort(MPI_COMM_WORLD, 3);
    combine(&local, work[t].best);
}
collect(local, rank); /* Solo el hilo inicial llama a MPI. */
MPI_Finalize();
```

El hilo inicial crea y espera a los trabajadores, combina resultados y ejecuta las colectivas.

Compilar y ejecutar el ejemplo pequeño

```
cd ../PMD-Repaso-Paralelismo
make examples
./build/serial
./build/pthreads
OMP_NUM_THREADS=3 ./build/openmp
mpirun -np 3 ./build/mpi
OMP_NUM_THREADS=2 mpirun -np 3 ./build/mpi_openmp
mpirun -np 3 ./build/mpi_pthreads
```

Resultado esperado

Distancia 13; camino $0 \to 2 \to 1 \to 3 \to 4 \to 5$.

Comparación de modelos

Modelo	Unidad	Datos	Combinación
Serial	tarea	locales	mínimo serial
Pthreads	hilo	memoria compartida	tras `join`
OpenMP	iteración	memoria compartida	tras barrera
MPI	proceso	memoria separada	`MINLOC` + `Bcast`
MPI + hilos	proceso e hilo	local por proceso	local, luego MPI

El reparto por primera arista es sencillo, pero puede producir desequilibrio de carga.

Ejercicios: corrección y reparto

1. Dibuje el árbol para las primeras dos decisiones. ¿Por qué basta explorar caminos simples?

Ejercicios: corrección y reparto

1. Dibuje el árbol para las primeras dos decisiones. ¿Por qué basta explorar caminos simples?
2. Elimine una arista y calcule manualmente el nuevo resultado; confirme con todas las versiones.

Ejercicios: corrección y reparto

1. Dibuje el árbol para las primeras dos decisiones. ¿Por qué basta explorar caminos simples?
2. Elimine una arista y calcule manualmente el nuevo resultado; confirme con todas las versiones.
3. Cambie el grafo para que el destino sea inalcanzable. ¿Participan todos los rangos en las colectivas?

Ejercicios: corrección y reparto

1. Dibuje el árbol para las primeras dos decisiones. ¿Por qué basta explorar caminos simples?
2. Elimine una arista y calcule manualmente el nuevo resultado; confirme con todas las versiones.
3. Cambie el grafo para que el destino sea inalcanzable. ¿Participan todos los rangos en las colectivas?
4. Para $P = 3$, escriba los índices de tarea de cada rango. ¿Qué ocurre con más procesos que tareas?

De un ejemplo pequeño a un grafo desde archivo

- Cada programa recibe archivo, origen y destino; la búsqueda conserva DFS exhaustivo y caminos simples.

De un ejemplo pequeño a un grafo desde archivo

- Cada programa recibe archivo, origen y destino; la búsqueda conserva DFS exhaustivo y caminos simples.
- Archivo de texto: primera línea V E; luego E líneas con `origen destino peso`, una arista dirigida por línea.

De un ejemplo pequeño a un grafo desde archivo

- Cada programa recibe archivo, origen y destino; la búsqueda conserva DFS exhaustivo y caminos simples.
- Archivo de texto: primera línea V E; luego E líneas con `origen destino peso`, una arista dirigida por línea.
- Vértices numerados desde 0; pesos enteros positivos, sin aristas duplicadas ni lazos; máximo 64 vértices.

De un ejemplo pequeño a un grafo desde archivo

- Cada programa recibe archivo, origen y destino; la búsqueda conserva DFS exhaustivo y caminos simples.
- Archivo de texto: primera línea V E; luego E líneas con `origen destino peso`, una arista dirigida por línea.
- Vértices numerados desde 0; pesos enteros positivos, sin aristas duplicadas ni lazos; máximo 64 vértices.
- El grafo se carga una vez y es de solo lectura durante la búsqueda; se guarda como listas de adyacencia.

Formato del archivo

```
4 4
0 1 2
0 2 5
1 2 1
2 3 3
```

- Ejemplo: 4 vértices, 4 aristas dirigidas; camino $0 \to 1 \to 2 \to 3$, costo 6.

Formato del archivo

```
4 4
0 1 2
0 2 5
1 2 1
2 3 3
```

- Ejemplo: 4 vértices, 4 aristas dirigidas; camino $0 \to 1 \to 2 \to 3$, costo 6.
- Para un grafo no dirigido, escriba ambas direcciones de cada arista.

Formato del archivo

```
4 4
0 1 2
0 2 5
1 2 1
2 3 3
```

- Ejemplo: 4 vértices, 4 aristas dirigidas; camino $0 \to 1 \to 2 \to 3$, costo 6.
- Para un grafo no dirigido, escriba ambas direcciones de cada arista.
- Se comprueban cabecera, rangos, pesos y aristas repetidas.

Leer y validar las aristas

```
for (int i = 0; i < g.m; ++i) {
    int u, v, w;
    if (fscanf(f, "%d%d%d", &u, &v, &w) != 3 ||
        u < 0 || u >= g.n || v < 0 || v >= g.n || u == v ||
        w <= 0 || g.degree[u] >= MAXV) {
        fputs("Invalid edge\n", stderr); exit(EXIT_FAILURE);
    }
    for (int j = 0; j < g.degree[u]; ++j)
        if (g.adj[u][j].to == v) { fputs("Duplicate edge\n", stderr); exit(EXIT_FAILURE);
  }
    g.adj[u][g.degree[u]++] = (Edge){v, w};
}
```

Leer y validar las aristas

```
for (int i = 0; i < g.m; ++i) {
    int u, v, w;
    if (fscanf(f, "%d%d%d", &u, &v, &w) != 3 ||
        u < 0 || u >= g.n || v < 0 || v >= g.n || u == v ||
        w <= 0 || g.degree[u] >= MAXV) {
        fputs("Invalid edge\n", stderr); exit(EXIT_FAILURE);
    }
    for (int j = 0; j < g.degree[u]; ++j)
        if (g.adj[u][j].to == v) { fputs("Duplicate edge\n", stderr); exit(EXIT_FAILURE);
  }
    g.adj[u][g.degree[u]++] = (Edge){v, w};
}
```

La cabecera se valida antes del bucle; después se comprueba que no queden datos adicionales.

Grafo de prueba: 58 vértices

- Archivo `data/capas_58.txt`: 58 vértices y 357 aristas dirigidas.

Grafo de prueba: 58 vértices

- Archivo `data/capas_58.txt`: 58 vértices y 357 aristas dirigidas.
- Origen 0, destino 57; entre ellos hay ocho capas de siete vértices cada una.

Grafo de prueba: 58 vértices

- Archivo `data/capas_58.txt`: 58 vértices y 357 aristas dirigidas.
- Origen 0, destino 57; entre ellos hay ocho capas de siete vértices cada una.
- Cada vértice se conecta a todos los de la siguiente capa: 7^8 = 5.764.801 caminos de 9 aristas.

Grafo de prueba: 58 vértices

- Archivo `data/capas_58.txt`: 58 vértices y 357 aristas dirigidas.
- Origen 0, destino 57; entre ellos hay ocho capas de siete vértices cada una.
- Cada vértice se conecta a todos los de la siguiente capa: 7^8 = 5.764.801 caminos de 9 aristas.
- Pesos reproducibles entre 1 y 13; distancia mínima obtenida: 13.

Grafo de prueba: 58 vértices

- Archivo `data/capas_58.txt`: 58 vértices y 357 aristas dirigidas.
- Origen 0, destino 57; entre ellos hay ocho capas de siete vértices cada una.
- Cada vértice se conecta a todos los de la siguiente capa: 7^8 = 5.764.801 caminos de 9 aristas.
- Pesos reproducibles entre 1 y 13; distancia mínima obtenida: 13.
- Siete tareas iniciales: una por cada arista saliente del origen; se exploran todos los caminos, sin poda.

Pthreads: cola de trabajo

```
static void *worker(void *arg) {
    Worker *w = arg; Queue *q = w->queue; w->best = empty_result();
    for (;;) {
        pthread_mutex_lock(&q->lock);
        int i = q->next++;
        pthread_mutex_unlock(&q->lock);
        if (i >= q->g->degree[q->source]) break;
        combine(&w->best, search_task(q->g, q->source, q->target, i));
    }
    return NULL;
}
```

Pthreads: cola de trabajo

```
static void *worker(void *arg) {
    Worker *w = arg; Queue *q = w->queue; w->best = empty_result();
    for (;;) {
        pthread_mutex_lock(&q->lock);
        int i = q->next++;
        pthread_mutex_unlock(&q->lock);
        if (i >= q->g->degree[q->source]) break;
        combine(&w->best, search_task(q->g, q->source, q->target, i));
    }
    return NULL;
}
```

El mutex protege el índice compartido. Cada trabajador conserva su mínimo privado; el hilo principal los combina tras `pthread_join`.

OpenMP: tareas desde el archivo

```
Result results[MAXV];
#pragma omp parallel for schedule(dynamic, 1)
for (int i = 0; i < g.degree[source]; ++i)
    results[i] = search_task(&g, source, target, i);
for (int i = 0; i < g.degree[source]; ++i)
    combine(&best, results[i]);
```

OpenMP: tareas desde el archivo

```
        Result results[MAXV];
        #pragma omp parallel for schedule(dynamic, 1)
        for (int i = 0; i < g.degree[source]; ++i)
            results[i] = search_task(&g, source, target, i);
        for (int i = 0; i < g.degree[source]; ++i)
            combine(&best, results[i]);
```

Una posición de resultado por arista del origen. El reparto dinámico asigna una iteración a la vez y la barrera implícita precede a la combinación.

Compilar y ejecutar el grafo desde archivo

```
make examples
./build/serial data/capas_58.txt 0 57
./build/pthreads data/capas_58.txt 0 57
OMP_NUM_THREADS=4 OMP_DYNAMIC=FALSE ./build/openmp data/capas_58.txt 0 57
make check
make benchmark
```

Las tres versiones imprimen detalles del grafo, distancia, camino y tiempo de búsqueda. Pthreads usa cuatro hilos; OpenMP utiliza la variable de entorno.

Benchmark: comparación de tiempos

Mediana de siete ejecuciones

Modelo	Tiempo (s)	Aceleración
Serial	0.044881	1.00 ×
Pthreads (4 hilos)	0.014777	3.04 ×
OpenMP (4 hilos)	0.015329	2.93 ×

- Equipo: Apple M1 Pro, Linux aarch64, GCC 16.1.1; cuatro hilos, compilación `-O2`; una ejecución de calentamiento y siete mediciones secuenciales por modelo.

Benchmark: comparación de tiempos

Mediana de siete ejecuciones

Modelo	Tiempo (s)	Aceleración
Serial	0.044881	1.00 ×
Pthreads (4 hilos)	0.014777	3.04 ×
OpenMP (4 hilos)	0.015329	2.93 ×

- Equipo: Apple M1 Pro, Linux aarch64, GCC 16.1.1; cuatro hilos, compilación `-O2`; una ejecución de calentamiento y siete mediciones secuenciales por modelo.
- Tiempo de pared monotónico: incluye creación y sincronización de hilos; excluye lectura del archivo e impresión.

Benchmark: comparación de tiempos

Mediana de siete ejecuciones

Modelo	Tiempo (s)	Aceleración
Serial	0.044881	1.00 ×
Pthreads (4 hilos)	0.014777	3.04 ×
OpenMP (4 hilos)	0.015329	2.93 ×

- Equipo: Apple M1 Pro, Linux aarch64, GCC 16.1.1; cuatro hilos, compilación `-O2`; una ejecución de calentamiento y siete mediciones secuenciales por modelo.
- Tiempo de pared monotónico: incluye creación y sincronización de hilos; excluye lectura del archivo e impresión.
- La aceleración es mediana serial dividida por la mediana del modelo. Es una medición de este equipo, no una garantía.

Ejercicios: grafo desde archivo

1. Cambie el número de hilos de OpenMP y el valor `THREADS` de Pthreads; mida 1, 2 y 4 hilos.

Ejercicios: grafo desde archivo

1. Cambie el número de hilos de OpenMP y el valor `THREADS` de Pthreads; mida 1, 2 y 4 hilos.
2. Añada la carga de archivos a MPI: ¿cómo distribuiría el grafo y cómo verificaría el camino ganador?

Ejercicios: grafo desde archivo

1. Cambie el número de hilos de OpenMP y el valor `THREADS` de Pthreads; mida 1, 2 y 4 hilos.
2. Añada la carga de archivos a MPI: ¿cómo distribuiría el grafo y cómo verificaría el camino ganador?
3. Compare con Dijkstra.